

Cognome.....Nome.....

Matricola.....Turno.....

Algoritmi e Strutture Dati

Appello del 24-04-2013 – Dott. Leporati / Prof. Zandron

Esercizio 1

Scrivere un algoritmo in pseudocodice che utilizzi la tecnica **divide-et-impera** per risolvere il seguente problema. Dati in ingresso:

- un **albero binario** T (non necessariamente di ricerca),
- un **intero positivo** p ,

l'algoritmo deve calcolare e restituire al chiamante il *numero di nodi* di T che si trovano ad una profondità **maggiore o uguale** di p . (Ricordiamo che la profondità di un nodo in un albero è la lunghezza del cammino che va dalla radice al nodo stesso).

Scrivere poi l'**equazione di ricorrenza** che esprime il tempo di calcolo dell'algoritmo *nel caso in cui l'albero T sia completo*, e **risolverla** utilizzando un metodo a piacere.

Esercizio 2

Scrivere un algoritmo **iterativo** che, ricevute in input due **pila** P e Q (ciascuna contenente dei numeri interi positivi, ordinati in modo tale che l'elemento più piccolo si trovi in cima alla pila) e una **pila** S (vuota), metta in S tutti gli elementi *che compaiono sia in P che in Q , evitando di inserire in S un elemento già presente* (gli elementi di S risulteranno quindi tutti diversi tra loro) *e facendo in modo che gli elementi di S risultino anch'essi ordinati* (con l'elemento più piccolo in cima alla pila).

Per agire sulle pile usare solo le operazioni **Push**, **Pop**, **Top** e **StackEmpty**. Come eventuali strutture dati d'appoggio si possono utilizzare unicamente **pila**.

Detti poi n ed m il numero di elementi presenti in P e in Q , rispettivamente, valutare i **tempi di calcolo**, espressi in funzione di n ed m , nel *caso migliore* e nel *caso peggiore* (specificando quando si verificano tali casi). In particolare, si indichi quante volte viene eseguita ogni operazione in pseudocodice, assumendo che ogni singola operazione elementare impieghi lo stesso tempo (costante).

Esempio: se P contiene la sequenza 2, 3, 3, 3, 5, 7, 7, 10, 12, 19, 20, 20 (dove 2 è l'elemento in cima alla pila), e Q contiene la sequenza 2, 2, 4, 5, 5, 6, 12, 12, 15 (dove anche in questo caso 2 è l'elemento in cima alla pila), al termine dell'esecuzione dell'algoritmo S dovrà contenere la sequenza 2, 5, 12 (dove, ancora una volta, 2 è l'elemento in cima alla pila).

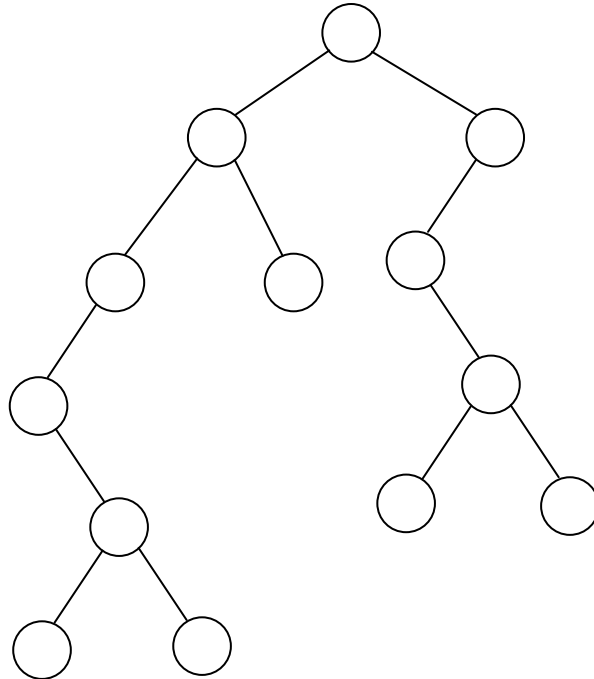
Esercizio 3

Data la sequenza di numeri interi:

26, 29, 9, 10, 3, 7, 17, 4, 8, 12, 19, 20, 23

riempire la struttura ad albero data qui sotto in modo tale che risulti un **albero binario di ricerca**. Si mostrino poi:

- le sequenze di interi ottenute visitando l'albero negli ordini **anticipato**, **simmetrico** e **posticipato** (ovvero, rispettivamente: pre-ordine, in-ordine e post-ordine);
- gli alberi ottenuti cancellando prima la chiave 3 e poi la chiave 17 (mostrare la struttura dell'albero ottenuta dopo *ogni singola cancellazione*, a partire dall'albero precedente). Spiegare brevemente come vengono fatte le operazioni di cancellazione.



Esercizio 4

Simulare l'esecuzione dell'algoritmo HEAPSORT sul seguente array:

$A = [14, 20, 6, 4, 18, 16]$

illustrando i vari passaggi, e mostrando lo stato dell'array **dopo ogni operazione di scambio** fra i suoi elementi. In particolare, si mostrino anzitutto gli scambi effettuati dalla funzione BUILDHEAP per risistemare gli elementi del vettore in modo da formare uno heap, e successivamente si mostrino gli scambi effettuati dall'algoritmo di ordinamento vero e proprio per ordinare gli elementi in ordine crescente. Se lo si ritiene opportuno, l'array può essere rappresentato in forma di albero; in tal caso, sul primo albero si indichino in maniera chiara gli indici degli elementi.